AriadneMem: Threading the Maze of Lifelong Memory for LLM Agents

Wenhui Zhu[1*], **Xiwen Chen**[2*], **Zhipeng Wang**[3*], **Jingjing Wang**[4], **Xuanzhao Dong**[1], **Minzhou Huang**[5], **Rui Cai**[6], **Hejian Sang**[7], **Hao Wang**[4], **Peijie Qiu**[8], **Yueyue Deng**[9], **Prayag Tiwari**[10], **Brendan Hogan Rappazzo**[2], **Yalin Wang**[1]

[1]Arizona State University, [2]Morgan Stanley, [3]Rice University, [4]Clemson University, [5]Northwestern University, [6]UC Davis, [7]Iowa State University, [8]Washington University in St. Louis, [9]Columbia University, [10]Halmstad University
wzhu59@asu.edu, xiwen.chen@morganstanley.com

Abstract

Long-horizon LLM agents require memory systems that remain accurate under fixed context budgets. However, existing systems struggle with two persistent challenges in long-term dialogue: (i) **disconnected evidence**, where multi-hop answers require linking facts distributed across time, and (ii) **state updates**, where evolving information (e.g., schedule changes) creates conflicts with older static logs. We propose **AriadneMem**, a structured memory system that addresses these failure modes via a decoupled two-phase pipeline. In the **offline construction phase**, AriadneMem employs *entropy-aware gating* to filter noise and low-information message before LLM extraction and applies *conflict-aware coarsening* to merge static duplicates while preserving state transitions as temporal edges. In the **online reasoning phase**, rather than relying on expensive iterative planning, AriadneMem executes *algorithmic bridge discovery* to reconstruct missing logical paths between retrieved facts, followed by *single-call topology-aware synthesis*. On LoCoMo experiments with GPT-4o, AriadneMem improves **Multi-Hop F1 by 15.2%** and **Average F1 by 9.0%** over strong baselines. Crucially, by offloading reasoning to the graph layer, AriadneMem reduces **total runtime by 77.8%** using only **497** context tokens. The code is available at https://github.com/LLM-VLM-GSL/AriadneMem.

1 Introduction

Large Language Model (LLM) agents operating in persistent, open-ended environments require robust long-term memory to maintain state consistency and perform multi-step reasoning over time. While large context windows have expanded, they remain insufficient for lifelong interaction and suffer from "lost-in-the-middle" phenomena Liu et al. (2024). Consequently, external memory systems have become a standard component of agent architecture, typically following a Retrieve-Augmented Generation (RAG) paradigm Lewis et al. (2020). However, within this paradigm, existing approaches face a fundamental trade-off between *structure* (how efficiently information is stored) and *connectivity* (how effectively scattered facts are linked).

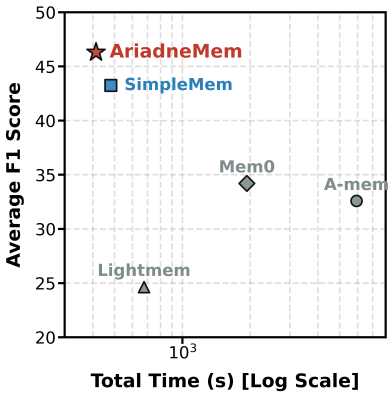

Figure 1: Efficiency-Performance Trade-off on LoCoMo benchmark.

Prior works navigate this trade-off through diverging strategies. Some systems rely on (i) raw-log re-

*Equal contribution

trieval Lewis et al. (2020), which preserves context but lacks structural connectivity. To bridge gaps in retrieval, others employ (ii) iterative reasoning Du et al. (2025), "simulating" connectivity by generating multiple queries and performing repeated search loops. While effective, this iterative process incurs significant latency and token costs Packer et al. (2023). Representing the current state-of-the-art in storage efficiency, SimpleMem Liu et al. (2026) shifts the paradigm by compressing dialogues into context-independent atomic entries. These entries function as self-contained semantic units where pronominal ambiguity is resolved and relative time is grounded to absolute timestamps (e.g., converting "He'll meet Bob tomorrow" to "Alice meets Bob on 2025-11-16"). This maximizes information density and reduces ambiguity.

Crucially, however, SimpleMem occupies a precarious middle ground. While it significantly improves on the information density of Strategy (i), it inadvertently retains its topological flatness, failing to resolve the challenge of *disconnected evidence*. Because it treats memory as a set of isolated atoms without intrinsic links, SimpleMem is forced to fall back on the expensive planning loops of Strategy (ii) to perform multi-hop reasoning (e.g., $A \to B \to C$), invoking the LLM to deduce intermediate "bridge" nodes (B) that lack direct semantic overlap.

Consequently, a distinct paradox emerges: While SimpleMem is highly efficient in terms of storage tokens, it suffers from significant interaction latency for complex queries. The reliance on runtime LLM inference to *bridge* disjoint facts effectively negates its speed advantage during multi-hop reasoning, introducing inference overhead precisely when the agent needs to think deepest.

Compounding this latency is a fundamental fragility regarding *state updates*: Inheriting the limitations of Strategy (i), the system lacks explicit temporal structure, struggling to distinguish between redundant repetition and evolving information (e.g., "meeting at 2pm" $\to$ "changed to 3pm").

In this work, we argue that the limitation lies in the *representation* itself. Relying on the LLM to implicitly reconstruct logical chains from flat fragments is both computationally expensive and prone to error. The key observation is that complex questions are rarely answered by isolated facts, but by connected *chains* of memories. To simultaneously eliminate retrieval-time latency and ground multi-hop inference in explicit topology, long-term memory should not be a flat set, but an evolutionary graph that directly encodes state transitions and entity relationships. We introduce **AriadneMem**[1] (overview of the framework is presented in Fig. 2), a framework that transforms memory retrieval from a probabilistic guessing game into a deterministic structural traversal.

Specifically, AriadneMem employs entropy-aware graph coarsening to manage memory density, merging semantically redundant nodes while strictly preserving state updates. Crucially, leveraging this graph structure enables us to execute multi-hop planning via Approximate Steiner Tree retrieval. Instead of a flat list, this process constructs a compact *query-specific evidence graph*, automatically discovering "bridge nodes" to connect scattered evidence. Within this subgraph, we mine explicit multi-hop paths (via DFS) which are serialized to guide the generator. This structured context enables AriadneMem to perform a single LLM call for final answer synthesis, thereby circumventing the expensive iterative planning loops required by prior methods.

Our contributions are summarized as follows:

- **From Iterative Planning to Structural Traversal:** We identify a fundamental efficiency bottleneck in existing agents: the reliance on expensive LLM reasoning to bridge disjoint memories. We propose to shift this burden to a graph-native layer, utilizing *algorithmic bridge discovery* and *bounded-depth path mining* to deterministically reconstruct evidence chains. This transition reduces interaction latency by **77.8%** while elevating multi-hop reasoning accuracy.

[1]Named after the Greek mythological figure Ariadne, who gave Theseus a thread to navigate the Labyrinth. Analogously, our system provides a structural *thread* (reasoning path) for agents to navigate the complex maze of lifelong memory.

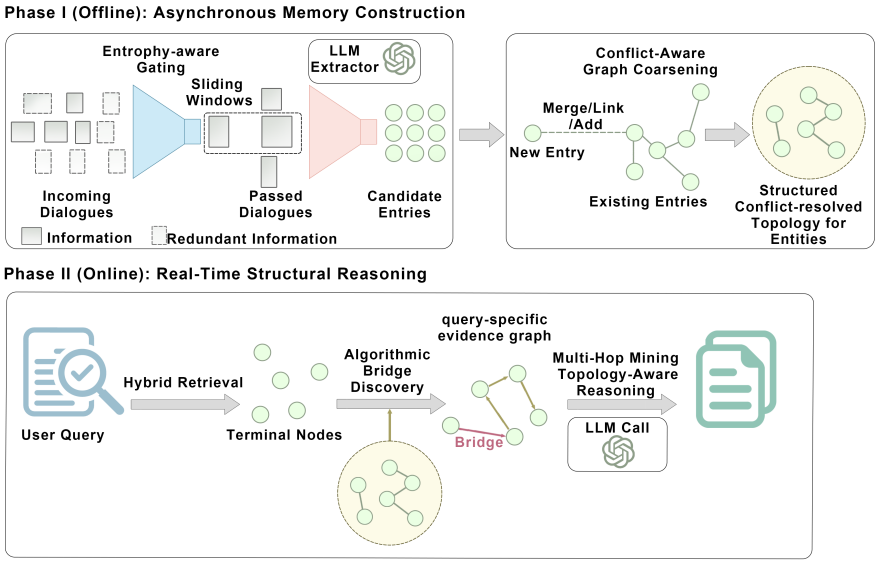

Figure 2: **Overview of the AriadneMem architecture.** The pipeline is decoupled into two phases: (I) Offline Memory Construction, which maintains an evolutionary graph via entropy-aware gating and conflict-aware coarsening to resolve state updates; and (II) Online Structural Reasoning, which connects disjoint evidence through algorithmic bridge discovery and performs topology-aware synthesis.

- **Entropy-Aware Evolutionary Memory:** Unlike static vector stores that suffer from information redundancy or catastrophic forgetting of state updates, AriadneMem maintains an evolving memory graph. By introducing *conflict-aware coarsening*, we merge redundant semantics while explicitly encoding state transitions as temporal links, ensuring the agent maintains a consistent "world model" of the dialogue history.
- **Topology-Aware Contextualization:** We present a novel serialization paradigm that injects the structural properties of the retrieved subgraph directly into the LLM. By providing *path-oriented grounding* rather than a flat list of fragments, AriadneMem effectively mitigates the "lost-in-the-middle" phenomenon and ensures high-fidelity answer synthesis within a compact context budget (avg. **497** tokens).
- **Superior Performance:** Extensive experiments on the LoCoMo benchmark demonstrate that AriadneMem achieves a significant leap in both reasoning quality and operational efficiency. We report a **15.2% improvement in Multi-Hop F1** and a **9.0% gain in Average F1** over the current SOTA, establishing AriadneMem as a highly practical framework for lifelong LLM agents. See Fig. 1 for performance-efficiency trade-off across different models.

2 Method

2.1 Problem Setup & Pipeline Overview

We consider a stream of dialogues $D = \{d_t\}_{t=1}^{T}$ where each item is $d_t = \langle s_t, x_t, t_t \rangle$ (speaker, text, optional timestamp). The system stores *atomic entries* $\mathcal{M} = \{m_k\}$, each with a lossless restatement S_k, keyword set $\mathcal{K}_k$, metadata record $\mathcal{R}_k$ (persons/entities/location/time) Liu et al. (2026), and dense embedding vector $\mathbf{v}_k \in \mathbb{R}^d$. Entries are indexed in a multi-view store: a semantic (dense) index for similarity search and a lexical (sparse) index for keyword matching. Given a user query q, the system retrieves a subgraph $G_q \subset \mathcal{M}$ to synthesize the

final answer a:

$$a = \mathrm{LLM}\big(q, \mathrm{Serialize}(G_q)\big). \tag{1}$$

AriadneMem implements a pipeline decoupled into two asynchronous phases, aligned with Figure 2:

- **Phase I (Offline): Asynchronous Memory Construction.** To process the continuous stream efficiently, incoming dialogues are subject to *entropy-aware gating* to filter low-information inputs before passing through *atomic extraction* to produce structured entries. The resulting facts are then *coarsened* into an evolutionary graph, where the system merges redundant duplicates while preserving state updates (e.g., changing schedules) as explicit temporal links.
- **Phase II (Online): Real-Time Structural Reasoning.** Upon receiving a query, we first check *fast paths* (cache and regex lookup) for immediate answers. If unresolved, we perform *hybrid retrieval* to find entry points, followed by *algorithmic bridge discovery* to connect disjoint evidence chains into a **query-specific evidence graph**. Finally, we perform *single-call topology-aware synthesis*, serializing this graph to guide the LLM in generating a multi-hop answer in one pass.

2.2 Phase I: Asynchronous Memory Construction

This phase transforms the raw stream D into a sparse, conflict-resolved evolutionary graph. We formally define an atomic entry $m \in \mathcal{V}$ as a tuple $m = \langle \mathbf{v}, \mathcal{K}, \mathrm{Ent}, t \rangle$, containing a dense embedding $\mathbf{v} \in \mathbb{R}^d$, a keyword set $\mathcal{K}$, extracted entities Ent, and timestamp t.

Entropy-Aware Gating. To prevent the memory store from being flooded with trivial chit-chat, we employ a pre-extraction gating mechanism. Let $E(\cdot)$ be the embedding function. For an incoming dialogue d_t, we retrieve its nearest neighbor $m^*(d_t)$ in the existing memory and compute a redundancy score r_t:

$$m^*(d_t) = \arg\max_{m \in \mathcal{V}} \cos\big(E(x_t), \mathbf{v}_m\big), \quad r_t = \cos\big(E(x_t), \mathbf{v}_{m^*}\big). \tag{2}$$

We apply a non-linear gating decision $\Phi_{\mathrm{gate}}(d_t)$ that blocks short-term repetition but allows long-term recurrence:

$$\Phi_{\mathrm{gate}}(d_t) = \begin{cases} 0, & r_t > \lambda_{\mathrm{red}} \ \wedge \ \Delta(t_t, t_{m^*}) < \delta_{\mathrm{short}}, \\ 1, & \text{otherwise}, \end{cases} \tag{3}$$

where λ_{red} is the redundancy threshold and δ_{short} is a short-horizon window (e.g., 1 hour) to filter near-immediate repetitions. If $\Phi_{\mathrm{gate}}(d_t) = 0$, we drop the input immediately.

Atomic Entry Extraction. For dialogues that pass gating ($\Phi = 1$), we invoke the LLM extractor $\mathcal{F}_\theta$ over a sliding window W_t.

$$\{m_k\} = \mathcal{F}_\theta(W_t). \tag{4}$$

This step produces a set of candidate atomic entries. By placing the gating mechanism *before* extraction, AriadneMem reduces redundant extraction calls compared to pipelines that extract from every dialogue turn.

Conflict-Aware Graph Coarsening. Simply appending new entries leads to linear growth in storage. We coarsen the candidate entries to maintain a sparse topology by distinguishing *redundancy* from *state updates*. For a new entry m and an existing entry $\tilde{m}$, we compute semantic similarity (sim) and keyword overlap (ovlp):

$$\mathrm{sim}(m, \tilde{m}) = \cos(\mathbf{v}_m, \mathbf{v}_{\tilde{m}}), \quad \mathrm{ovlp}(m, \tilde{m}) = \frac{|\mathcal{K}_m \cap \mathcal{K}_{\tilde{m}}|}{\max(1, |\mathcal{K}_m|)}. \tag{5}$$

We classify the relationship into three distinct actions using thresholds λ_{coal} and λ_{ovlp}:

$$\mathrm{Action} = \begin{cases} \textsc{Merge}, & \text{if } \mathrm{sim} > \lambda_{\mathrm{coal}} \wedge \mathrm{ovlp} > \lambda_{\mathrm{ovlp}} \quad \text{(Duplicate)} \\ \textsc{Link}, & \text{if } \mathrm{sim} > \lambda_{\mathrm{coal}} \wedge \mathrm{ovlp} \leq \lambda_{\mathrm{ovlp}} \quad \text{(State Update)} \\ \textsc{Add}, & \text{otherwise.} \end{cases} \tag{6}$$

Specifically, **Merge** discards true duplicates and updates the timestamp of $\tilde{m}$. Crucially, **Link** handles cases where semantics align but details differ (e.g., "meeting at 2pm" vs "meeting at 3pm"); here, we retain m and create a directed temporal edge $\tilde{m} \to m$, explicitly preserving the state transition fidelity. Finally, **Add** inserts distinct inputs as new isolated nodes.

Outcome: Graph as a Navigational Prior. By the end of Phase I, the raw stream has been transformed into a structured, conflict-resolved topology. This offline construction serves a dual purpose for the online phase: (1) **Bridge Enablement:** The established temporal and entity links provide the necessary "roads" for the Steiner Tree algorithm (Phase II) to discover hidden connections between disjoint facts; and (2) **State Resolution:** By explicitly modeling updates as directed edges (e.g., 2pm $\to$ 3pm), the graph resolves potential paradoxes before retrieval, ensuring the agent acts on the latest state without needing to reason over conflicting raw logs.

2.3 Phase II: Real-Time Structural Reasoning

Upon receiving a query q, AriadneMem executes a synchronous pipeline to construct a query-specific evidence graph G_q and synthesize an answer a. We reformulate this process as an algorithmic search problem rather than a generative planning task.

Fast Paths (Heuristic Short-Circuiting). Before launching general-purpose retrieval, we check two lightweight shortcuts that do not require additional LLM calls: (i) an *enhanced cache* for common query patterns built during ingestion (e.g., maintaining a running counter for "How many emails from Alice?"), and (ii) a regex-based *attribute lookup* for simple "X's attribute" questions (e.g., directly mapping "What is Bob's phone number?" to metadata fields). If either path returns sufficient evidence, the system directly constructs a minimal graph containing the lookup facts, bypassing the heavier retrieval steps.

Hybrid Retrieval. For complex queries, we first identify a set of *terminal nodes* V_{term} via hybrid retrieval (dense + lexical):

$$V_{\text{term}} = \text{Top-}k_{\text{sem}}(q) \ \cup \ \text{Top-}k_{\text{lex}}(q). \tag{7}$$

Consistent with multi-view retrieval principles, we use $E(q)$ for dense similarity and keyword matching for lexical retrieval. We treat k_{sem} and k_{lex} as hyperparameters. Additionally, we extract target entities from q using simple heuristics to downweight candidates that lack entity alignment.

Base Graph Construction. We construct a base graph $G_0 = (V_{\text{term}}, E_0)$ to establish initial connectivity for the query-specific evidence graph. Let $\text{Ent}(m)$ denote the set of entities extracted for node m. We define a directed edge $m_i \to m_j$ if nodes share entities or are temporally close:

$$\text{Ent}(m_i) \cap \text{Ent}(m_j) \neq \varnothing \quad \text{or} \quad |t_i - t_j| < \delta_{\text{time}}. \tag{8}$$

In our reference implementation, we set $\delta_{\text{time}} = 6$ hours. This strict window encourages a "narrative backbone," linking events that likely belong to the same immediate context while keeping the graph sparse.

Algorithmic Bridge Discovery (Steiner Tree Approximation). Terminal nodes are often topologically disconnected. To recover these links without retrieval-time planning loops, we approximate a Steiner Tree by searching for *bridge nodes* b^*. For disconnected pairs $(m_i, m_j) \in V_{\text{term}}$, we query the memory for a node that maximizes semantic connectivity within the valid time interval:

$$b^* = \underset{m \in \mathcal{V} \setminus V_{\text{term}}}{\arg\max} \ \cos\big(E(\mathbf{q}_{ij}), \mathbf{v}_m\big) \ \ \text{s.t.} \ \ t_m \in [\min(t_i, t_j), \max(t_i, t_j)]. \tag{9}$$

Here, the bridge query $\mathbf{q}_{ij}$ is constructed by concatenating the entities and keywords of the endpoints: $\text{Ent}(m_i) \cup \text{Ent}(m_j) \cup \mathcal{K}_i \cup \mathcal{K}_j$. To maintain precision, we attempt bridge search only when the time gap is moderate (1–168 hours) and strictly consider only the top-5 candidates from semantic search. If a valid bridge is found, it is added to the graph $(m_i \to b^* \to m_j)$.

Multi-Hop Path Mining & Node Budget. To provide structured guidance, we explicitly mine reasoning paths via Depth-First Search (DFS) on the augmented graph G_q. Let $\mathcal{P}_q$ be the set of directed paths up to length L (set to $L = 3$ hops in our implementation):

$$\mathcal{P}_q = \{p \in G_q \mid 2 \leq |p| \leq L, \text{temporally consistent}\}. \tag{10}$$

To control the context length for generation, we enforce a **node budget** (e.g., keeping between 8 and 25 nodes). We prioritize paths based on length and temporal coherence, pruning the graph if the budget is exceeded.

Topology-Aware Reasoning. Finally, we generate the answer by conditioning a single LLM call on the serialized topology. We serialize G_q and $\mathcal{P}_q$ into a textual format $\mathcal{C}_{\text{graph}}$, listing timestamped facts and explicit path indicators. Crucially, to ensure robustness on benchmarks like LoCoMo, we attach a set of **explicit answer rules**: (i) *Length Constraints* (concise vs. list based on query type); (ii) *Temporal Fidelity* (strict copying of timestamps, avoiding relative normalization); (iii) *Aggregation Logic* (specifying counting formats); and (iv) *Formatting* (JSON requirements as output). The final answer is synthesized via:

$$a = \text{LLM}(q, \mathcal{C}_{\text{graph}}). \tag{11}$$

This structured context enables AriadneMem to perform complex multi-hop reasoning in a single inference step, circumventing the latency of iterative generation. A qualitative example is presented in Fig. 3.

Algorithm 1 AriadneMem Pipeline: Asynchronous Memory & Reasoning

Require: Stream D, Memory $\mathcal{V}$, Index $\mathbb{I}$, Thresholds $\lambda_{\{\text{red,coal,ovlp}\}}$, Windows $\delta_{\{\text{short,time}\}}$
Ensure: Answer a for query q

Phase I: Asynchronous Memory (Offline)
for $d_t \in D$ **do**
  $m^* \leftarrow \text{NN}(x_t, \mathcal{V})$; $r_t \leftarrow \cos(E(x_t), \mathbf{v}_{m^*})$
  if $r_t > \lambda_{\text{red}} \wedge \Delta t < \delta_{\text{short}}$ **then continue** ▷ Gating Φ_{gate}
  Buffer d_t into W_t. **if** full:
    $\{m\} \leftarrow \mathcal{F}_\theta(W_t)$; **for** each m vs $\tilde{m} \in \mathcal{V}$:
      $\text{Op} \leftarrow (\text{sim} > \lambda_{\text{coal}})?\ (\text{ovlp} > \lambda_{\text{ovlp}}?\textsc{Merge} : \textsc{Link}) : \textsc{Add}$
      Execute Op (Update $\mathcal{V}, \mathbb{I}$; Link $\tilde{m} \to m$ if Link); Reset W_t
end for

Phase II: Structural Reasoning (Online)
$V_{\text{term}} \leftarrow \text{Top-}k_{\text{sem}}(q, \mathbb{I}) \cup \text{Top-}k_{\text{lex}}(q, \mathbb{I})$
$G_q \leftarrow (V_{\text{term}}, E)$ where $E = \{(u, v) \mid \text{Ent}_u \cap \text{Ent}_v \neq \varnothing \vee \Delta t < \delta_{\text{time}}\}$
for disconnected $(m_i, m_j) \in G_q$ **do**
  $\mathbf{q}_{ij} \leftarrow \text{Ent}_i \cup \text{Ent}_j \cup \mathcal{K}_i \cup \mathcal{K}_j$
  $b^* \leftarrow \arg\max_{m \in \mathcal{V} \setminus V_{\text{term}}} \cos(E(\mathbf{q}_{ij}), \mathbf{v}_m)$ s.t. $t_m \in [t_i, t_j]$
  if b^* found **then** $G_q \leftarrow G_q \cup \{m_i \to b^* \to m_j\}$ ▷ Steiner Approx.
end for
$\mathcal{C}_{\text{graph}} \leftarrow \text{Serialize}(\text{DFS}(G_q, L))$ s.t. Node Budget
return $\text{LLM}(q, \mathcal{C}_{\text{graph}})$

3 Related Work

Memory Systems for LLM Agents. Recent approaches manage memory through virtual context or structured representations. Virtual context methods such as MemGPT Packer et al. (2023) provide paging and controller-style memory, but often store raw logs, inducing redundancy and increasing processing cost. Structured memory systems such as Mem0 Dev & Taranjeet (2024); Chhikara et al. (2025), A-Mem Xu et al. (2025), and LightMem Fang et al. (2026) improve coherence and retrieval but can still preserve referential and temporal ambiguities if the stored text remains minimally processed. SimpleMem Liu et al. (2026) addresses this by semantic structured compression into context-independent atomic entries, plus query-aware retrieval to improve token efficiency. Compared to structured-memory

Question: What musical artists/bands has Melanie seen?
[Ground Truth: Summer Sounds, Matt Patterson]

Facts:
...
...
[**F11**] 2023-07-20: Melanie and her family went to the beach recently and the kids had a blast.
[**F12**] 2023-08-13: Melanie celebrated her daughter's birthday with a concert by Matt Patterson.
[**F13**] 2023-08-24: Melanie spent the day with her family volunteering at a homeless shelter yesterday.
[**F14**] 2023-08-27: Melanie took her kids to a park yesterday.
[**F15**] 2023-08-28: Melanie mentioned she is a fan of both classical music like Bach and Mozart, as well as modern music like Ed Sheeran's 'Perfect'
[**F16**] 2023-08-28: Melanie stated that she plays the clarinet and started when she was young.
[**F17**] 2023-08-28: Melanie mentioned that the band 'Summer Sounds' played an awesome pop song that got everyone dancing and singing.
...
...

Reasoning Path: ...
...
F3 → F18 → F19
F18 → F19 → F20
F15 → F16 → F17
...
Final Answer:
Matt Patterson, Summer Sounds

Figure 3: **Qualitative Example of Structural Reasoning.** A sample output showing how AriadneMem retrieves and serializes a coherent, timestamped evidence chain to answer a multi-hop question.

baselines such as SimpleMem Liu et al. (2026), AriadneMem focuses on constructing a connected evidence subgraph rather than returning an unstructured top-k list.

Context Management and Retrieval Efficiency. Beyond storage, efficient access to historical information is a core challenge. Retrieval-augmented generation (RAG) Lewis et al. (2020) decouples memory from inference, but flat top-k retrieval can miss intermediate evidence required by multi-hop questions. Graph-based RAG variants (e.g., GraphRAG Edge et al. (2024)) build structured summaries for query-focused retrieval, mainly targeting static knowledge corpora. In long-term episodic memory, evidence must additionally preserve temporal flow and state updates. AriadneMem addresses these challenges by (i) conflict-aware coarsening for updates and (ii) approximate Steiner completion with bridge-node discovery to construct connected evidence subgraphs.

Compression and Token Efficiency. Prompt compression methods (e.g., LLMLingua Pan et al. (2024)) reduce token usage but may lose task-critical details when applied post hoc. SimpleMem Liu et al. (2026) emphasizes write-time semantic lossless compression, ensuring each memory entry is self-contained. AriadneMem focuses on retrieval-time structure: rather than expanding to large contexts or running iterative planning loops, we retrieve a compact connected subgraph and expose explicit reasoning paths to the generator.

4 Experiments

We evaluate AriadneMem on long-term conversational memory benchmarks such as LoCoMo loc (2024) to answer three questions. First, does AriadneMem improve long-horizon QA accuracy compared to prior agent memory systems, with emphasis on MultiHop and Temporal subsets. Second, does it improve efficiency when we measure both retrieved context length and runtime, including retrieval time and end-to-end time. Third, which components account for the gains, as measured by ablation and retrieval-depth sensitivity. Following the reporting style of SimpleMem Liu et al. (2026), we report per-subset F1 and BLEU together with Token Cost in Table 1, runtime breakdown in Table 2, and ablations and sensitivity in Tables 3 and 4.

4.1 Implementation Details

We follow the LoCoMo evaluation protocol and report results on MultiHop, Temporal, OpenDomain, and SingleHop subsets. Within each block of Table 1, all methods are evaluated under the same underlying LLM backbone. We report F1 and BLEU for answer quality. We also report token cost, defined as the number of tokens in the retrieved memory context fed to the answer generator. In our main experiments, the window size is 20, the redundancy and coarsening thresholds are $\lambda_{\text{red}} = 0.6$ and $\lambda_{\text{coal}} = 0.7$, and the recall depths are $k_{\text{sem}} = 20$ and $k_{\text{lex}} = 5$.

4.2 Main Results and Analysis

We evaluate AriadneMem across multiple LLM backbones and compare against recent memory baselines, including LoCoMo loc (2024), ReadAgent Lee et al. (2024), MemoryBank Zhong et al. (2024), MemGPT Packer et al. (2023), A-Mem Xu et al. (2025), LightMem Fang et al. (2026), Mem0 Dev & Taranjeet (2024); Chhikara et al. (2025), and SimpleMem Liu et al. (2026). Table 1 reports the breakdown by question type (MultiHop, Temporal, OpenDomain, SingleHop) and the associated Token Cost.

Overall accuracy. AriadneMem attains the highest Average F1 for all evaluated backbones. On GPT-4o, AriadneMem reaches 42.57 Average F1, compared to 39.06 for SimpleMem and 36.09 for Mem0. On GPT-4.1-mini, AriadneMem reaches 46.30, compared to 43.24 for SimpleMem and 34.20 for Mem0. On Qwen3-Plus, AriadneMem reaches 46.03, compared to 37.49 for SimpleMem and 35.85 for Mem0. These results indicate that the connected-evidence retrieval improves accuracy beyond flat retrieval under the same backbone.

MultiHop and Temporal. MultiHop is where AriadneMem shows its largest gains. On GPT-4o, AriadneMem reaches 41.34 MultiHop F1, compared to 35.89 for SimpleMem and 35.13 for Mem0. On Qwen3-Plus, AriadneMem reaches 42.17, compared to 33.74 for SimpleMem and 32.42 for Mem0. On GPT-4.1-mini, AriadneMem reaches 44.24, compared to 43.46 for SimpleMem and 30.14 for Mem0. Temporal follows a similar trend. For example, on GPT-4o AriadneMem reaches 57.94 Temporal F1, compared to 56.71 for SimpleMem and 52.38 for Mem0. These results support the design choice of connecting scattered evidence with bridge nodes and then conditioning generation on the resulting subgraph. Temporal matters because it reflects timeline ordering and state updates across sessions. On Qwen3-Plus, AriadneMem reaches 63.67 Temporal F1, compared to 50.87 for SimpleMem. On GPT-4.1-mini, AriadneMem reaches 64.28, compared to 58.62 for SimpleMem. This aligns with conflict-aware coarsening for updates and time-aware graph completion, and it is reinforced by the generator's temporal-fidelity constraints.

Token cost. On GPT-4o, AriadneMem uses 497 tokens, compared to 550 for SimpleMem and 985 for Mem0. On Qwen3-Plus, AriadneMem uses 460 tokens, compared to 583 for SimpleMem and 1,020 for Mem0. On GPT-4.1-mini, AriadneMem uses 916 tokens, compared to 531 for SimpleMem. Overall, AriadneMem keeps Token Cost below full-context baselines while improving accuracy, and the remaining variation reflects the node budget and how much intermediate evidence is needed for a query.

Table 1: Performance on the LoCoMo benchmark with High-Capability Models (GPT-4.1 series, GPT-4o, and Qwen3-Plus).

Model	Method	MultiHop F1	MultiHop BLEU	Temporal F1	Temporal BLEU	OpenDomain F1	OpenDomain BLEU	SingleHop F1	SingleHop BLEU	Average F1	Average BLEU	Token Cost
GPT-4.1-mini	LoCoMo	25.02	21.62	12.04	10.63	19.05	17.07	18.68	15.87	18.70	16.30	16,910
	ReadAgent	6.48	5.6	5.31	4.23	7.66	6.62	9.18	7.91	7.16	6.09	643
	MemoryBank	5.00	4.68	5.94	4.78	5.16	4.52	5.72	4.86	5.46	4.71	**432**
	MemGPT	17.72	16.02	19.44	16.54	11.29	10.18	25.59	24.25	18.51	16.75	16,977
	A-Mem	25.06	17.32	51.01	44.75	13.22	14.75	41.02	36.99	32.58	28.45	2,520
	LightMem	24.96	21.66	20.55	18.39	19.21	17.68	33.79	29.66	24.63	21.85	612
	Mem0	30.14	27.62	48.91	44.82	16.43	14.94	41.3	36.17	34.20	30.89	973
	SimpleMem	43.46	**38.82**	58.62	50.10	19.76	18.04	51.12	43.53	43.24	37.62	531
	AriadneMem	**44.24**	37.04	**64.28**	**52.33**	**22.56**	**18.86**	**54.11**	**48.65**	**46.30**	**39.22**	916
GPT-4o	LoCoMo	28.00	18.47	9.09	5.78	16.47	14.80	61.56	54.19	28.78	23.31	16,910
	ReadAgent	14.61	9.95	4.16	3.19	8.84	8.37	12.46	10.29	10.02	7.95	805
	MemoryBank	6.49	4.69	2.47	2.43	6.43	5.30	8.28	7.10	5.92	4.88	569
	MemGPT	30.36	22.83	17.29	13.18	12.24	11.87	40.16	36.35	25.01	21.06	16,987
	A-Mem	32.86	23.76	39.41	31.23	17.10	15.84	44.43	38.97	33.45	27.45	1,216
	LightMem	28.15	21.83	36.53	29.12	13.38	11.54	33.76	28.02	27.96	22.63	645
	Mem0	35.13	27.56	52.38	44.15	17.73	15.92	39.12	35.43	36.09	30.77	985
	SimpleMem	35.89	32.83	56.71	20.57	18.23	16.34	45.41	39.25	39.06	27.25	550
	AriadneMem	**41.34**	**34.49**	**57.94**	**46.34**	**25.02**	**22.93**	**45.97**	**40.11**	**42.57**	**35.97**	**497**
Qwen3-Plus	LoCoMo	24.15	18.94	16.57	13.28	11.81	10.58	38.58	28.16	22.78	17.74	16,910
	ReadAgent	9.52	6.83	11.22	8.15	5.41	5.23	9.85	7.96	9.00	7.04	742
	MemoryBank	5.25	4.94	1.77	6.26	5.88	6.00	6.90	5.57	4.95	5.69	**302**
	MemGPT	25.80	17.50	24.10	18.50	9.50	7.80	40.20	42.10	24.90	21.48	16,958
	A-Mem	26.50	19.80	46.10	35.10	11.90	11.50	43.80	36.50	32.08	25.73	1,427
	LightMem	28.95	24.13	42.58	38.52	16.54	13.23	40.78	36.52	32.21	28.10	606
	Mem0	32.42	21.24	47.53	39.82	17.18	14.53	46.25	37.52	35.85	28.28	1,020
	SimpleMem	33.74	29.04	50.87	43.31	18.41	16.24	46.94	38.16	37.49	31.69	583
	AriadneMem	**42.17**	**36.06**	**63.67**	**49.84**	**24.46**	**22.92**	**53.80**	**48.71**	**46.03**	**39.38**	460

Table 2: Efficiency comparison on LoCoMo loc (2024). We report construction time, retrieval time, total time, and Average F1, here Baseline methods are based on GPT-4.1-mini.

Method	Construction Time (s)	Retrieval Time (s)	Total Time(s)	Average F1
A-mem	5140.5s	796.7s	5937.2s	32.58
Lightmem	97.8s	577.1s	675.9s	24.63
Mem0	1350.9s	583.4s	1934.3s	34.20
SimpleMem	92.6s	388.3s	480.9s	43.24
AriadneMem w GPT-4o	**38.0s**	391.9s	429.9s	42.57
AriadneMem w GPT-4.1-mini	113.5s	**299.7s**	**413.2s**	**46.30**

4.3 Efficiency Comparison

We compare efficiency across memory systems along three axes: retrieval latency, retrieved context length (Token Cost), and end-to-end runtime. We report **retrieval time** as the wall-clock time for the retriever to return evidence, excluding the final answer generation call. Table 2 reports runtime breakdown. Compared to Mem0, AriadneMem reduces total time from 1934.3s to 429.9s (GPT-4o) and increases Average F1 from 34.20 to 42.57. Compared to SimpleMem, AriadneMem reduces total time from 480.9s to 429.9s (GPT-4o), with Average F1 42.57 vs. 43.24. Under GPT-4.1-mini, AriadneMem reaches 46.30 Average F1 at 413.2s total time. Taken together, Table 2 shows that AriadneMem reduces total runtime relative to iterative retrieval baselines while keeping accuracy competitive with strong structured-memory systems.

4.4 Ablation Study

We ablate key components of AriadneMem to quantify their contribution to accuracy and efficiency. In the final version, we will report both (i) accuracy (Avg. F1; and optionally per-category F1) and (ii) efficiency (Token Cost / retrieval time) for each ablation.

Impact of Entropy-Aware Gating. Removing gating reduces Average F1 from 42.57 to 41.68, a drop of 0.89. This indicates that gating helps efficiency and contributes modestly to accuracy under this setup.

Table 3: Ablation study by question type on LoCoMo loc (2024) with GPT-4o as the backbone.

Configuration	MultiHop F1	MultiHop BLEU	Temporal F1	Temporal BLEU	OpenDomain F1	OpenDomain BLEU	SingleHop F1	SingleHop BLEU	Avg. F1
Full AriadneMem	**41.34**	**34.49**	**57.94**	**46.34**	**25.02**	**22.93**	**45.97**	**40.11**	**42.57**
w/o Entropy-aware gating	41.12	33.03	55.72	44.28	24.32	21.64	45.58	39.81	41.68
w/o Coarsening	40.48	33.06	53.93	42.95	24.15	22.13	47.11	41.66	41.42
w/o bridge discovery	35.95	25.55	44.64	32.25	20.51	15.17	41.19	35.27	35.57
w/o topology-aware reasoning	33.67	24.79	45.66	33.56	23.97	18.66	41.79	35.77	36.27

Table 4: Comparison of retrieval depth sensitivity with GPT-4o as the backbone.

(a) k_{sem}

k	Avg. F1	Retrieval Time (s)
1	27.51	265.3s
10	39.59	392.3s
20	**42.57**	**391.9s**
30	40.27	276.8s

(b) k_{lex}

k	Avg. F1	Retrieval Time (s)
1	42.25	385.9s
3	42.40	392.7s
5	**42.57**	**391.9s**
10	41.16	288.7s

Impact of Conflict-Aware Coarsening. Removing coarsening reduces Average F1 from 42.57 to 41.42, a drop of 1.15. The per-category changes differ by subset. SingleHop increases from 45.97 to 47.11, while Temporal decreases from 57.94 to 53.93. This suggests coarsening trades redundancy reduction against preserving fine-grained updates.

Impact of Bridge Discovery. Removing bridge discovery reduces Average F1 from 42.57 to 35.57, a drop of 7.00. The largest drops are on MultiHop, which goes from 41.34 to 35.95, and Temporal, which goes from 57.94 to 44.64. This supports the use of bridge nodes to connect evidence that is missed by initial hybrid recall.

Impact of Topology-Aware Reasoning. Removing topology-aware reasoning reduces Average F1 from 42.57 to 36.27, a drop of 6.30. MultiHop decreases from 41.34 to 33.67 and Temporal decreases from 57.94 to 45.66. This indicates that passing the same facts without structural guidance is not sufficient for multi-evidence questions. Overall, Table 3 shows that bridge discovery and topology-aware reasoning are the two dominant components for MultiHop and Temporal accuracy under GPT-4o.

Table 4 varies retrieval depth. Increasing k_{sem} improves Average F1 from 27.51 at k=1 to 42.57 at k=20. Varying k_{lex} yields smaller changes in Average F1. Average F1 is 42.25 at k=1 and 42.57 at k=5. Retrieval time is not monotonic in k because node limiting and graph construction can dominate runtime.

5 Conclusion

In this paper, we presented AriadneMem, a structured memory system designed to thread the complex maze of lifelong memory for LLM agents. By decoupling the memory pipeline into an offline construction phase and an online reasoning phase, we effectively address the persistent challenges of disconnected evidence and state updates in long-term dialogues. AriadneMem leverages a conflict-aware coarsening mechanism to maintain a dynamic, evolving world model through structured temporal edges. By introducing bridge discovery and DFS-based path mining, we transform the disjointed fragments of long-horizon contexts into coherent reasoning chains, effectively resolving the challenges of multi-hop information retrieval in disconnected evidence scenarios.

In summary, AriadneMem achieves a superior balance of efficiency and accuracy. By replacing iterative planning with structural graph traversal, it reduces runtime by 77.8% while boosting Multi-Hop F1 by 15.2%. With its compact token footprint, AriadneMem provides a fast, accurate, and scalable memory foundation for long-horizon LLM agents.

References

Locomo: Long-term conversational memory benchmark. https://github.com/snap-research/locomo, 2024.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *ArXiv*, abs/2504.19413, 2025.

Khant Dev and Singh Taranjeet. mem0: The memory layer for ai agents. https://github.com/mem0ai/mem0, 2024.

Xingbo Du, Loka Li, Duzhen Zhang, and Le Song. Memr ^{3}:: Memory retrieval via reflective reasoning for llm agents. *arXiv preprint arXiv:2512.20237*, 2025.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

Jizhan Fang, Xinle Deng, Haoming Xu, Ziyan Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, et al. Lightmem: Lightweight and efficient memory-augmented generation. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=dyJ0GWpjJB.

Kuang-Huei Lee, Xinyun Chen, Hiroki Furuta, John Canny, and Ian Fischer. A human-inspired reading agent with gist memory of very long contexts. *arXiv preprint arXiv:2402.09727*, 2024.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. Simplemem: Efficient lifelong memory for llm agents. *arXiv preprint arXiv:2601.02553*, 2026.

Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12:157–173, 2024.

Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph Gonzalez. Memgpt: Towards llms as operating systems. *ArXiv*, abs/2310.08560, 2023. URL https://api.semanticscholar.org/CorpusID:263909014.

Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, et al. Llmlingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. *arXiv preprint arXiv:2403.12968*, 2024.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *ArXiv*, abs/2502.12110, 2025. URL https://api.semanticscholar.org/CorpusID:276421617.

Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 19724–19731, 2024.